AI Declaration Form

1.

I acknowledge the use of [1] ChatGPT[2](https://chatgpt.com/c/6814df7b-6794-8012-af21-b51f20741d3b) to generate materials that were included within my final assessment in modified form. On 28 April 2025, I entered the following prompt [3]. [4] The output was then modified further to better represent my own tone and style of writing.

[3]

> help me generate unit test code for Santorini Boardgame in Java, based on 5 functionalities: 1. Movement of worker 2. Building of worker 3. initialise board 4. winning condition 5. move and build with god cards (Artemis and Demeter)

[4]

> Here's a Java unit test template using **JUnit 5** that covers the five core functionalities of a **Santorini** board game implementation. This assumes you already have relevant classes like `Board`, `Worker`, `Player`, `GodCard`, and methods like `move()`, `build()`, `initializeBoard()`, `checkWin()`, etc.
>
> ---
>
> ### ✅ Assumptions
>
> To proceed, we assume the following:
>
> - `Worker` can move and build.
> - `Board` is a 5x5 grid.
> - `GodCard` is an abstract class or interface; `Artemis` and `Demeter` are subclasses.
> - `Player` has two `Worker`s and an optional `GodCard`.
> - Tiles can have a height (0-3) and a dome.
>
> ---
>
> ### 🧪 Unit Test Code

java

CopyEdit

```
import org.junit.jupiter.api.BeforeEach;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;


public class SantoriniGameTest {


    private Board board;

    private Player player;

    private Worker worker;


    @BeforeEach

    public void setUp() {

        board = new Board();

        board.initializeBoard();

        player = new Player("Alice");

        worker = new Worker(player, 0, 0); // starts at (0, 0)

        player.addWorker(worker);

        board.placeWorker(worker, 0, 0);

    }


    @Test
```

```
    public void testWorkerMovement() {
        assertTrue(board.moveWorker(worker, 1, 1));
        assertEquals(1, worker.getX());
        assertEquals(1, worker.getY());
    }

    @Test
    public void testWorkerBuild() {
        board.moveWorker(worker, 1, 1);
        assertTrue(board.build(worker, 1, 2));
        assertEquals(1, board.getTile(1, 2).getHeight());
    }

    @Test
    public void testBoardInitialization() {
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                assertEquals(0, board.getTile(i, j).getHeight());
                assertFalse(board.getTile(i, j).hasDome());
                assertNull(board.getTile(i, j).getWorker());
            }
        }
    }
```

```
@Test
public void testWinningCondition() {
    board.getTile(1, 1).setHeight(2);
    board.placeWorker(worker, 1, 1);
    board.getTile(2, 2).setHeight(3);
    assertTrue(board.moveWorker(worker, 2, 2));
    assertTrue(board.checkWin(worker));
}

@Test
public void testMoveWithArtemis() {
    Artemis artemis = new Artemis();
    player.setGodCard(artemis);
    artemis.setWorker(worker);

    // First move
    assertTrue(artemis.move(worker, 1, 0));
    // Second move (not back to original position)
    assertTrue(artemis.move(worker, 2, 0));
    assertEquals(2, worker.getX());
    assertEquals(0, worker.getY());
}
```

```
    @Test
    public void testBuildWithDemeter() {
        Demeter demeter = new Demeter();
        player.setGodCard(demeter);
        demeter.setWorker(worker);
        board.moveWorker(worker, 1, 1);

        // First build
        assertTrue(demeter.build(worker, 1, 2));
        assertEquals(1, board.getTile(1, 2).getHeight());

        // Second build (not on same space)
        assertTrue(demeter.build(worker, 2, 2));
        assertEquals(1, board.getTile(2, 2).getHeight());
    }
}
```

📌 Notes

- You'll need to adapt this test if your method or class names differ.
- This setup assumes god card behavior is encapsulated in the card (i.e., `Artemis.move()` and `Demeter.build()`).

- Ensure your methods like `moveWorker()`, `build()`, and `checkWin()` return booleans for test assertions.

Would you like me to provide class skeletons for `Board`, `Worker`, or the god cards too?

2.

I acknowledge the use of [1] ChatGPT[2](https://chatgpt.com/c/6814df7b-6794-8012-af21-b51f20741d3b) to generate materials that were included within my final assessment in modified form. On 29 April 2025, I entered the following prompt [3]. [4] The output was then modified further to better represent my own tone and style of writing.

[3]

Give me a detail explanation of SOLID principle, include example in Java code too.

[4]

The **SOLID principles** are a set of five design principles intended to make software designs more understandable, flexible, and maintainable. The acronym stands for:

Single Responsibility Principle
Open/Closed Principle
Liskov Substitution Principle
Interface Segregation Principle
Dependency Inversion Principle

🔹 1. Single Responsibility Principle (SRP)

Definition: A class should have only **one reason to change**, meaning it should only have one job.

✅ Java Example:

java
CopyEdit

```
// Bad design: one class handles both user and logging
class UserManager {
    public void addUser(String name) {
        // logic to add user
```

```
        System.out.println("User added: " + name);
    }

    public void log(String message) {
        // logging logic
        System.out.println("Log: " + message);
    }
}

// Good design: separate logging into its own class
class Logger {
    public void log(String message) {
        System.out.println("Log: " + message);
    }
}

class UserManager {
    private Logger logger = new Logger();

    public void addUser(String name) {
        // logic to add user
        logger.log("User added: " + name);
    }
}
```

🔹 2. Open/Closed Principle (OCP)

Definition: Software entities should be **open for extension, but closed for modification**.

✅ Java Example:

java
CopyEdit

```
// Bad: modifying PaymentProcessor every time a new method is
added
class PaymentProcessor {
    public void pay(String type) {
        if (type.equals("paypal")) {
            // PayPal logic
```

```
        } else if (type.equals("creditcard")) {
            // Credit card logic
        }
    }
}

// Good: use polymorphism to extend functionality
interface Payment {
    void pay();
}

class PaypalPayment implements Payment {
    public void pay() {
        System.out.println("Paid via PayPal");
    }
}

class CreditCardPayment implements Payment {
    public void pay() {
        System.out.println("Paid via Credit Card");
    }
}

class PaymentProcessor {
    public void processPayment(Payment payment) {
        payment.pay();
    }
}
```

🔹 3. Liskov Substitution Principle (LSP)

Definition: Subtypes must be substitutable for their base types without altering the correctness of the program.

✅ Java Example:

java
CopyEdit

```
// Violation: Square overrides behavior of Rectangle
class Rectangle {
```

```
    protected int width, height;

    public void setWidth(int w) { width = w; }
    public void setHeight(int h) { height = h; }
    public int getArea() { return width * height; }
}

class Square extends Rectangle {
    public void setWidth(int w) {
        width = height = w;
    }
    public void setHeight(int h) {
        width = height = h;
    }
}

// Fix: Separate hierarchy
interface Shape {
    int getArea();
}

class RectangleShape implements Shape {
    private int width, height;
    public RectangleShape(int w, int h) {
        this.width = w;
        this.height = h;
    }
    public int getArea() {
        return width * height;
    }
}

class SquareShape implements Shape {
    private int side;
    public SquareShape(int s) {
        this.side = s;
    }
    public int getArea() {
        return side * side;
    }
}
```

🔹 4. Interface Segregation Principle (ISP)

Definition: No client should be forced to depend on methods it does not use.

✅ Java Example:

java
CopyEdit

```
// Bad: one fat interface
interface Worker {
    void work();
    void eat();
}

// Robot doesn't eat — forced to implement unnecessary method
class Robot implements Worker {
    public void work() { System.out.println("Robot working"); }
    public void eat() {} // unnecessary
}

// Good: split into smaller interfaces
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

class Human implements Workable, Eatable {
    public void work() { System.out.println("Human working"); }
    public void eat() { System.out.println("Human eating"); }
}

class Robot implements Workable {
    public void work() { System.out.println("Robot working"); }
}
```

🔹 5. Dependency Inversion Principle (DIP)

Definition: High-level modules should not depend on low-level modules. Both should depend on abstractions.

✅ Java Example:

java
CopyEdit

```
// Bad: High-level module depends on low-level
class Keyboard {
    public String getInput() { return "Typing..."; }
}

class Computer {
    private Keyboard keyboard = new Keyboard();

    public void type() {
        System.out.println(keyboard.getInput());
    }
}

// Good: depend on abstraction
interface InputDevice {
    String getInput();
}

class Keyboard implements InputDevice {
    public String getInput() {
        return "Typing...";
    }
}

class Mouse implements InputDevice {
    public String getInput() {
        return "Clicking...";
    }
}

class Computer {
    private InputDevice inputDevice;
```

```
    public Computer(InputDevice inputDevice) {
        this.inputDevice = inputDevice;
    }

    public void useInput() {
        System.out.println(inputDevice.getInput());
    }
}
```

🧠 Summary Table:

Principle	Purpose	Key Idea
SRP	Maintainability	One class = one job
OCP	Extensibility	Use inheritance/interfaces
LSP	Correctness	Subtypes shouldn't break expectations
ISP	Focused design	Prefer small, specific interfaces
DIP	Flexibility	Depend on abstractions, not concrete classes

Would you like a diagram or UML illustration of any of these examples?

3.

I acknowledge the use of [1] ChatGPT[2](https://chatgpt.com/c/6814df7b-6794-8012-af21-b51f20741d3b) to generate materials that were included within my final assessment in modified form. On 1 May 2025, I entered the following prompt [3]. [4] The output was then modified further to better represent my own tone and style of writing.

[3]

Give me ideas on how to write test cases for Santorini based on the 5 functionalities

[4]

Writing effective test cases for your **Santorini** board game based on the five core functionalities requires identifying **expected behaviors**, **invalid inputs**, **edge cases**, and **special rules** (like god cards). Below are test case ideas categorized by each functionality, with example scenarios.

✅ 1. Worker Movement

Goal: Verify that workers can move to valid tiles, and are restricted appropriately.

✔️ Test Case Ideas:

Test Case	Description	Expected Result
Move to adjacent empty tile	Move worker from (2,2) to (2,3)	Success
Move to occupied tile	Target tile has another worker	Fail
Move to tile with dome	Tile has dome built	Fail
Move more than 1 tile away	E.g., from (2,2) to (4,4)	Fail
Move up more than 1 level	From height 0 to 2	Fail
Move down any number of levels	From height 3 to 1	Success

✅ 2. Worker Building

Goal: Ensure that workers can build appropriately, and restrictions apply.

✔️ Test Case Ideas:

Test Case	Description	Expected Result
Build on adjacent empty tile	Worker at (1,1) builds on (1,2)	Success
Build on non-adjacent tile	Build at (3,3) while at (1,1)	Fail
Build on tile with dome	Try building on a domed tile	Fail
Build on tile with max height (3)	Should place dome instead	Success (dome added)
Build on tile with worker	Attempt to build on occupied tile	Fail

---

✅ 3. Initialize Board

Goal: Confirm that board starts in a clean, valid state.

✔️ Test Case Ideas:

Test Case	Description	Expected Result
All tiles are empty	After initialization	No workers on any tile
All tiles have height 0	No pre-built structures	Pass
No domes	Board has no domes	Pass
5x5 dimensions	Board grid size	Pass

✅ 4. Winning Condition

Goal: Validate that win condition (move from level 2 to 3) is detected correctly.

✔️ Test Case Ideas:

Test Case	Description	Expected Result
Move from level 2 to 3	Worker climbs to level 3	Win triggered
Move from level 1 to 2	Normal move	No win
Build level 3 and move down	Should not trigger win	No win
Move using Artemis to level 3	Validate god card movement win	Win (if rules allow)
Win with Demeter active	After build-move	Win if legal conditions r

---

✅ 5. God Card Logic (Artemis & Demeter)

Goal: Ensure god powers function as described.

👼 Artemis (Move twice, not back to original tile)

Test Case	Description	Expected Result
Move once to (1,0), then (2,0)	Not back to start	Success
Move back to original tile	e.g., (0,0) → (1,0) → (0,0)	Fail

Move only once	Optional second move	Success

🌾 Demeter (Build twice on different tiles)

Test Case	Description	Expected Result
Build on two different tiles	e.g., (1,2) and (2,2)	Success
Build on same tile twice	Not allowed	Fail
Build once only	Optional second build	Success

📌 General Test Case Design Tips:

- Test **valid** and **invalid** actions.
- Use **edge cases** like border tiles (e.g., (0,0), (4,4)).
- Test **god powers interactions** with base mechanics.
- Create **integration test scenarios** (e.g., full move+build+win cycle).
- Include **performance tests** if using AI or rule engines.

Would you like me to help write these as actual **JUnit test case methods**?

Legend: [1] AI system(s); [2] link to the tool; [3] how the tool was used or prompts included; [4] output included in the task.